

IoT-Enabled Smart Access Control System with Offline Log Synchronization

High-Level Software Architecture:

The system is designed using a **Finite State Machine (FSM)** architecture, where each state is responsible for a single operation. This modular approach separates user interaction, authentication, networking, and error handling, making the system easier to debug, maintain, and extend.

The application consists of the following major modules:

❖ **Authentication Module**

- Performs Role-Based Access Control (RBAC) using a hard coded defined user table.
- Validates entered IDs and retrieves the associated role.

❖ **User Interface Module**

- Handles keypad input.
- Displays prompts, access status, and role information on the I2C LCD.

❖ **Networking Module**

- Connects to WiFi.
- Obtains real-time timestamps using NTP.
- Updates authenticated access events as JSON to online log using HTTP POST.

❖ **Offline Logging Module**

- Temporarily stores successful authentication events into a queue whenever the network is unavailable.
- Automatically uploads pending logs once connectivity is restored.

❖ **Emergency and Security Module**

- Handles emergency interrupt requests.
- Implements login timeout and lockout after repeated invalid attempts.

State Machine Logic:

The software is organized into the following states:

EMPTY

- Default IDLE state.
- Executes through the `cEmpty()` function.
- Waits for the first keypad digit.
- Starts a new authentication session, signals this by turning on the Yellow led.

CNC (Code Not Complete)

Responsible for collecting keypad input.

Functions performed:

- Reads keypad digits using the `processKey()` function.
- Supports backspace (*) using the `removeDigit()` function.
- Masks the entered passcode (**).
- Detects input timeout (7 seconds) by the `codeEntryTimeout()` function.
- Calls authentication via the `verifyCode()` function when four digits are entered and then moves to the subsequent state.

CIC (Code Incorrect)

Executed after an invalid authentication attempt.

Responsibilities:

- Displays "Incorrect" and "Try Again".
- Updates strike counter.
- Returns to **EMPTY** state.
- Enters lockout after three consecutive failures.

CC (Code Correct)

This state is responsible only for user feedback.

Responsibilities:

- Displays the user role.
- Displays "ACCESS GRANTED".
- Waits approximately 100 ms.
- Transitions to **UPLOADING** state.

Separating this from the upload operation ensures that networking is independent of the user interface.

UPLOADING

Responsible for recording successful authentication events, done using the `uploadLog()` function.

Operations performed:

- Retrieves current NTP timestamp using the `getTime()` function.
- Creates a JSON payload.
- Attempts an HTTP POST.

Two possible outcomes exist:

- **WiFi available**
 - Log is transmitted immediately using the `executePost()` function.
- **WiFi unavailable**
 - Log is stored locally inside the offline queue via `enqueueLog()` function.

The current implementation logs authenticated user access. In a production deployment, the same mechanism can be extended to log emergency override events as well.

After processing, the system returns to **EMPTY** state.

LOCKOUT

Entered after three invalid authentication attempts using `handleLockout()` function.

The system ignores further keypad input for 15 seconds before resetting the strike counter and returning to **EMPTY** state.

EMERGENCY

Triggered through an external interrupt and further taken care of using `handleEmergency()` function.

Displays an emergency override message before automatically returning to **EMPTY** state.

Offline Queue Design:

To satisfy the project requirement of retaining logs during network outages, a lightweight circular queue was implemented.

Each queued record contains:

```
struct Log
{
    String id;
    String time;
};
```

The queue has a fixed capacity of ten entries.

```
Log queue[10];
```

Three indices manage the queue:

- **front** – oldest stored record
- **rear** – next insertion location
- **count** – current number of stored logs

Operations provided include:

- `enqueueLog()`
- `dequeueLog()`
- `queueFull()`
- `queueEmpty()`

Queue Operation

When WiFi is unavailable:

Successful Login



`enqueueLog()`



Stored in RAM

I simulated an offline queue by temporarily assigning keys 'A' and 'B' on Keypad to have a function to disconnect WiFi and reconnect to WiFi.

```
HTTP Response Code: 200
POST Successful
ID: 6971, DATE-TIME: 2026-07-09 20:51:02
WiFi Disconnected
Stored Log. Queue Size: 1
Stored Log. Queue Size: 2
Reconnecting...
HTTP Response Code: 200
POST Successful
ID: 9821, DATE-TIME: 2026-07-09 20:51:24
Queued Log Uploaded
HTTP Response Code: 200
POST Successful
ID: 9822, DATE-TIME: 2026-07-09 20:51:29
Queued Log Uploaded
```

Automatic Recovery

The `flushQueue()` function executes continuously inside the main loop.

Whenever:

WiFi Connected

AND

Queue Not Empty

the oldest queued log is transmitted.

Only after a successful HTTP response is that entry removed from the queue.

To avoid overwhelming the server, uploads are rate-limited using a non-blocking timer (`millis()`), ensuring only one queued record is transmitted every 500 ms.

This satisfies the requirement of automatic synchronization after network restoration while maintaining non-blocking execution.

Memory Management Strategy:

The project was designed to minimize dynamic memory allocation on the ESP32.

Fixed-Size Storage

Instead of dynamically allocating log records, a fixed-size array was used:

```
Log queue[10];
```

This provides:

- Constant memory usage
- No runtime allocation
- Predictable RAM consumption

Circular Queue

Rather than shifting array elements after every dequeue operation, the queue uses circular indexing.

Advantages include:

- $O(1)$ insertion
- $O(1)$ deletion

- No unnecessary memory copies

ArduinoJson Usage

A new `JsonDocument` is created locally inside `executePost()` whenever a log is transmitted.

```
JsonDocument doc;
```

The JSON object exists only for the duration of the POST request and is automatically destroyed when the function returns.

This approach avoids retaining large JSON objects in global memory and minimizes heap usage.

The JSON payload itself contains only the required fields:

- User ID
- Timestamp

which keeps serialization overhead low.

Non-Blocking System Design:

The application avoids the use of `delay()`.

Instead, all timing behavior is implemented using `millis()`.

Examples include:

- Code entry timeout
- Lockout timer
- Access Granted display duration
- Upload interval for queued logs

This allows keypad input, emergency interrupts, LCD updates, and network recovery logic to operate concurrently without blocking program execution.